Perambulando - Documentação dos endpoints

Conferido em 18/10/2026 a partir dos controllers, DTOs e serviços salvos do projeto. Esta documentação descreve a implementação atual; não representa uma execução de testes no banco.

Convenções

- URL local usada no arquivo de testes: `http://localhost:5014`. Depende de como a API foi iniciada.
- Corpos de requisição são JSON: `Content-Type: application/json`.
- Endpoints autenticados exigem `Authorization: Bearer SEU_ACCESS_TOKEN`.
- `ADMIN ou HOST` significa possuir pelo menos uma dessas roles. A role não dispensa a verificação de propriedade nos serviços de edição e desativação.
- Os IDs de rota são inteiros. As actions validam que sejam maiores que zero; valores que não correspondem à restrição `int` não selecionam essas rotas.
- Respostas de erro dos controllers usam ProblemDetails ou ValidationProblemDetails. Falhas de autenticação/autorização do middleware podem não ter o mesmo corpo.
- Requisições protegidas podem retornar `401` por token ausente, inválido ou expirado e `403` quando faltar uma role exigida.
- `204` não possui corpo. Listas vazias podem retornar `200` com `[]`.
- Os exemplos usam a grafia atual da API: `instagranUrl`, `typeofhost`, `typeofUser` e `categoryIds`.

Inventário

Controller	Método	Caminho	Acesso
User	POST	`/api/User/auth/Register`	Público
User	POST	`/api/User/auth/Login`	Público
User	GET	`/api/User/Profile`	Autenticado
User	PUT	`/api/User/Edit`	Autenticado
User	DELETE	`/api/User`	Autenticado
Host	POST	`/api/Host/Create`	Autenticado
Host	GET	`/api/Host/GetAllHostsbyUserId`	ADMIN ou HOST
Host	GET	`/api/Host/Details/{hostId}`	Autenticado
Host	PUT	`/api/Host/{hostId}/Edit`	ADMIN ou HOST e dono
Host	PATCH	`/api/Host/{hostId}/Delete`	ADMIN ou HOST e dono
Programming	POST	`/api/Programming/{HostId}/Create`	ADMIN ou HOST e dono do host
Programming	GET	`/api/Programming/Details/{programmingId}`	Público
Programming	PUT	`/api/Programming/{hostId}/Edit/{Programmingid}`	ADMIN ou HOST e dono do host
Programming	PATCH	`/api/Programming/{hostId}/Delete/{programmingId}`	ADMIN ou HOST e dono do host
Category	GET	`/api/Category/GetallCategories`	Público

UserController

POST /api/User/auth/Register - Cadastrar usuário

Cria uma conta pelo ASP.NET Identity. Não exige login e não devolve JWT; após cadastrar, use o login.

Corpo (`UserRegisterDTO`):

```
{
  "name": "UsuarioTeste",
  "email": "usuario@example.com",
  "password": "SenhaForte123@!",
  "typeofUser": 0
}
```

- `typeofUser`: `0` = Local; `1` = Turist.

- Nome, e-mail e senha são encaminhados ao Identity. O DTO não possui validações explícitas por atributos.

- O e-mail deve ser único. Atualmente `name` é salvo em `UserName`, sujeito à unicidade e às restrições de nome do Identity.
- A senha é validada pelo Identity; não há personalização de suas regras no bloco de configuração atual.

Retornos: `201` com `{"message":"Usuário cadastrado com sucesso."}`; `400` para dados inválidos ou erros do Identity, como e-mail duplicado e senha rejeitada.

POST /api/User/auth/Login - Autenticar

Corpo (`UserLoginDTO`):

```
{
  "email": "usuario@example.com",
  "password": "SenhaForte123@!"
}
```

Ambos os campos são obrigatórios e o e-mail deve ter formato válido. O usuário precisa existir, estar ativo e passar na verificação de senha e nas condições de acesso do Identity.

Retornos: `200` com o objeto de autenticação abaixo; `400` para falha na validação do DTO; `401` para falha no login.

```
{
  "accessToken": "JWT",
  "tokenType": "Bearer",
  "expiresAt": "2026-10-18T20:00:00Z"
}
```

`expiresAt` é ilustrativo: a expiração real depende de `Jwt:ExpirationHours`.

GET /api/User/Profile - Consultar o próprio perfil

Exige JWT, sem corpo ou ID na URL. O usuário é identificado pelo token.

Retorna `200` com `id`, `name`, `email`, `typeofUser`, `isPro` e `createdAt`. Retorna `404` se o usuário não existir ou estiver inativo.

PUT /api/User/Edit - Editar o próprio usuário

Exige JWT e identifica o usuário pelo token. Substitui nome, e-mail e tipo; não altera senha nem roles.

```
{
  "name": "UsuarioAtualizado",
  "email": "usuario@example.com",
  "typeofUser": 1
}
```

Requisitos (`UserEditDTO`): `name` obrigatório, até 256 caracteres; `email` obrigatório, formato válido, até 256 caracteres; `typeofUser` obrigatório e pertencente ao enum. O nome e o e-mail são aparados no mapeamento e a alteração passa pelo Identity.

Retornos: `200` com `{"user": {"...": "perfil atualizado"}, "auth": {"...": "novo JWT"}}`; `400` para validação ou rejeição pelo Identity; `404` para usuário inexistente/inativo.

O frontend deve aplicar o token de `auth.accessToken` para refletir as claims atualizadas.

DELETE /api/User - Excluir a própria conta

Exige JWT. A action recebe `UserRegisterDTO`, mas usa o e-mail para localizar a conta e compara seu ID com o usuário do JWT.

Exemplo de corpo compatível com o DTO atual:

```
{
  "name": "UsuarioTeste",
  "email": "usuario@example.com",
  "password": "SenhaForte123@!",
  "typeofUser": 0
}
```

A action não verifica a senha enviada: a autorização é feita pelo JWT e pela comparação de propriedade. Os demais campos não são utilizados na exclusão.

Retornos: `204` após `UserManager.DeleteAsync`; `403` se o e-mail pertencer a outra conta; `404` se a conta não existir; `400` se o Identity rejeitar a exclusão. É exclusão pelo Identity, não desativação via `IsActive`.

HostController

POST /api/Host/Create - Cadastrar estabelecimento

Exige usuário autenticado e ativo. O `UserId` do dono é obtido do JWT; não é recebido no JSON. Uma conta pode cadastrar vários hosts.

Corpo (`HostRegisterDTO`):

```
{
  "typeofhost": 0,
  "cnpj": "12.345.678/0001-90",
  "logoUrl": "https://example.com/logo.png",
  "instagranUrl": "https://www.instagram.com/empresa",
  "address": "Rua de Teste, 100",
  "neighborhood": "Centro",
  "cep": "50000-000",
  "phone": "81999999999",
  "name": "Minha empresa",
  "description": "Descrição da empresa."
}
```

Tipos: `0` = Productor (Produtor); `1` = Gastronomy (Gastronomia); `2` = Business (Comércio).

O DTO de cadastro ainda não possui atributos que exijam os campos ou validem o enum. Não há validação de CNPJ/CEP nem geocodificação nesse fluxo. O host é salvo sem exigir programações.

O serviço atribui a role `HOST` ao usuário, caso necessário. A resposta contém um JWT atualizado, evitando repetir o login.

Retornos: `201` com `{"host":{"id":10,"name":"Minha empresa"},"auth":{"accessToken":"JWT","tokenType":"Bearer","expiresAt":"..."}}`; `401` para usuário ausente/inativo; `500` para as falhas tratadas de cadastro ou atribuição de role. Se a role falhar após a gravação, a resposta informa que o host já foi salvo.

GET /api/Host/GetAllHostsbyUserId - Listar meus hosts

Exige JWT com role `ADMIN` ou `HOST`. Sem corpo ou parâmetros. Retorna apenas hosts ativos cujo `UserId` corresponde ao usuário do token.

Retorno: `200` com lista de entidades `Hosts`, inclusive `[]`. Contém os campos do host, como `id`, `userId`, `typeofhost`, dados institucionais, endereço, `isActive`, `createdAt` e `updatedAt`. A action retorna entidades, não um DTO específico para a listagem.

GET /api/Host/Details/{hostId} - Consultar perfil da empresa

Exige JWT e `hostId` positivo. Sem corpo. Qualquer usuário autenticado pode consultar um host ativo; esta consulta não exige ser o dono.

Retornos: `200` com `HostDetailsDTO`; `400` para ID não positivo; `404` para host inexistente/inativo.

Campos: `id`, `name`, `description`, `typeofhost`, `cnpj`, `logoUrl`, `instagranUrl`, `address`, `neighborhood`, `cep`, `phone` e `programmings`.

`programmings` contém programações ativas ordenadas por início. Pode ser `[]`. O mapeamento atual usa `ProgrammingDetailsDTO`, mas não preenche suas categorias; também não inclui IDs das programações. A consulta não concede permissão para editar.

PUT /api/Host/{hostId}/Edit - Editar estabelecimento

Exige JWT com role `ADMIN` ou `HOST`, `hostId` positivo e ser dono do host ativo. Recebe os mesmos nomes de campos do exemplo de cadastro, usando `HostEditDTO`. É edição completa dos campos editáveis.

Campo	Requisito no edit
`typeofhost`	Obrigatório; enum válido
`name`	Obrigatório; máximo 150 caracteres
`description`	Obrigatório; máximo 2000
`address`	Obrigatório; máximo 300
`neighborhood`	Obrigatório; máximo 150
`cep`	Obrigatório; máximo 9
`cnpj`	Máximo 18
`phone`	Máximo 30
`logoUrl`, `instagranUrl`	Máximo 2048 cada

Retornos: `200` com a entidade `Hosts` atualizada; `400` para validação; `404` se o host estiver indisponível para o usuário. Atualiza `UpdatedAt` em UTC.

PATCH /api/Host/{hostId}/Delete - Desativar estabelecimento

Exige JWT com role `ADMIN` ou `HOST`, `hostId` positivo e ser dono do host ativo. Não recebe corpo.

Altera `IsActive` para `false` e atualiza `UpdatedAt`. Não remove o registro nem altera individualmente as programações; as consultas de programação deixam de apresentá-las por exigirem host ativo.

Retornos: `204`; `400` para ID não positivo; `404` para host inexistente, inativo ou de outro usuário. Uma repetição após desativar retorna `404`.

ProgrammingController

POST /api/Programming/{HostId}/Create - Cadastrar programação

Exige JWT com role `ADMIN` ou `HOST` e `HostId` positivo de um host ativo pertencente ao usuário do token.

Corpo (`ProgrammingCreateDTO`):

```
{
  "name": "Festival de música",
  "description": "Apresentações ao vivo.",
  "startDate": "2027-12-10T18:00:00Z",
  "endDate": "2027-12-10T22:00:00Z",
  "isFree": false,
  "address": "Rua do Evento, 200",
  "neighborhood": "Centro",
  "cep": "50000-000",
  "imageUrl": "https://example.com/evento.png",
  "ticketUrl": "https://example.com/ingressos",
  "instagranUrl": "https://www.instagram.com/evento",
  "minPrice": 30,
  "maxPrice": 60,
  "categoryIds": [1, 2]
}
```

Requisitos e comportamento:

- `name`, `description`, `address`, `neighborhood` e `cep` são obrigatórios pelo DTO.
- O início deve estar no futuro e ser anterior ao término. Use datas ISO 8601 em UTC com `Z`, compatíveis com a persistência das datas.
- Preços devem ser não negativos e o mínimo não pode superar o máximo. Para gratuita, ambos devem ser zero.
- `categoryIds` deve conter pelo menos um ID positivo. Categorias inexistentes geram erro tratado de chave estrangeira.
- `imageUrl`, `ticketUrl` e `instagranUrl` são opcionais no DTO. Não há validação de formato de URL por atributo.
- O endereço pode ser diferente do host. Não há limite de quantidade de programações implementado.
- Programação e vínculos de categorias são persistidos no mesmo `SaveChangesAsync`.
- Limitações atuais: IDs repetidos não são rejeitados no create; não há verificação de categoria ativa ao gravar. `IsFree` e preços são tipos não anuláveis: `[Required]` não detecta sua omissão e eles assumem `false`/`0`.

Retornos: `201` com entidade `Programming` e cabeçalho `Location` apontando para Details; `400` para as validações e categorias inexistentes; `404` para host inexistente, inativo ou de outro usuário.

GET /api/Programming/Details/{programmingId} - Consultar detalhes públicos

Público, inclusive para visitantes. Exige apenas `programmingId` positivo na URL. Sem corpo. A programação e seu host precisam estar ativos.

Retornos: `200` com `ProgrammingDetailsDTO`; `400` para ID não positivo; `404` para programação inexistente, inativa ou de host inativo.

Campos do retorno: `name`, `description`, `startDate`, `endDate`, `isFree`, `address`, `neighborhood`, `cep`, `imageUrl`, `ticketUrl`, `instagranUrl`, `hostName`, `hostLogoUrl`, `createdDate`, `updateDate`, `minPrice`, `maxPrice` e `categories`.

`categories` é uma lista das categorias vinculadas, com `id`, `name` e `isActive`. A consulta de vínculos não filtra categorias inativas. O DTO atual não inclui `id` da programação nem `hostId`.

PUT /api/Programming/{hostId}/Edit/{Programmingid} - Editar programação e categorias

Exige JWT com role `ADMIN` ou `HOST`, IDs positivos, host ativo do usuário e programação ativa pertencente ao host.

Recebe os mesmos campos do exemplo de create, usando `ProgrammingEditDTO`. É uma substituição completa dos campos editáveis, inclusive da seleção de categorias.

- Obrigatórios: `name`, `description`, `startDate`, `endDate`, `isFree`, `address`, `neighborhood`, `cep`, `minPrice`, `maxPrice` e `categoryIds`.
- Datas devem estar em UTC, e o início deve ser anterior ao término. O edit não possui a checagem de início futuro que existe no create.
- Preços devem ser finitos, não negativos e ordenados. Se gratuita, ambos devem ser zero.
- Categorias: ao menos uma, IDs positivos e sem repetição. IDs inexistentes geram `400`; não há validação de status ativo.
- A lista enviada é a seleção final: os vínculos retirados são removidos e os novos são adicionados. Os já existentes permanecem.
- A edição preserva a propriedade e a data de criação; atualiza `UpdateDate` em UTC. Campos e vínculos são salvos juntos.

Retornos: `200` com entidade `Programming` atualizada; `400` para validação ou categorias inexistentes; `404` quando host/programação estiver indisponível para o usuário.

PATCH /api/Programming/{hostId}/Delete/{programmingId} - Desativar programação

Exige JWT com role `ADMIN` ou `HOST`, IDs positivos, host ativo do usuário e programação ativa pertencente ao host. Sem corpo.

Altera `IsActive` para `false` e atualiza `UpdateDate`. Não remove o registro nem os vínculos de categorias.

Retornos: `204`; `400` para IDs não positivos; `404` para registro inexistente, inativo ou indisponível para o usuário/host. Uma repetição após desativar retorna `404`.

CategoryController

GET /api/Category/GetallCategories - Listar categorias ativas

Público, sem corpo ou parâmetros. Consulta apenas categorias com `IsActive = true`, ordenadas por ID.

Retorno: `200` com lista de objetos contendo `id`, `name` e `isActive`, inclusive `[]`. Use os IDs retornados em `categoryIds` ao criar ou editar programações.

Fluxo de uso

1. Cadastrar usuário e fazer login.
2. Enviar o JWT para criar o host.
3. Substituir o token local pelo `auth.accessToken` retornado no cadastro do host, que contempla a role HOST.
4. Consultar categorias e criar uma programação usando o ID do host.
5. Para gestão, listar os próprios hosts, consultar os detalhes do host e usar as rotas de edição/desativação.
6. Visitantes podem consultar os detalhes de programação diretamente, sem JWT.

O frontend pode remover o token ao sair. Não existe endpoint de logout ou refresh token nos controllers atuais. As respostas não incluem `canEdit`/`canDelete`.

Diferença encontrada no arquivo de testes

O arquivo `Projetos - 4.http` ainda contém `GET /api/Programming/{hostId}/GetAllProgrammingsbyHostId`, mas essa action não está no `ProgrammingController` salvo atualmente. Portanto, ela não integra os 15 endpoints documentados. A função existe nas camadas internas e é usada para listar programações dentro do perfil do host. Esta documentação não altera nem recria a action.

Fontes consultadas

No projeto `C:/Users/Artur/source/repos/Projetos - 4/Projetos - 4`: `Presentation/Controllers`, `Application/DTO`, os app services de User/Host/Programming, seus serviços e repositórios, `CategoryRepository`, `JwtTokenService` e a configuração de autenticação em `Presentation/Program.cs`.

Os retornos descritos são os fluxos previstos no código. Exceções não tratadas de banco/configuração podem produzir erros adicionais. Cadastro/edição de host e programação retornam entidades em alguns pontos; não devem ser interpretados como o mesmo contrato dos DTOs de detalhes.